

SmartMan 交付物说明

李孟霖 屈阳 刘治权 赵浦渊

1 数据简介

本项目所使用的基础语料库来源于开源项目 NL2Bash [1]。该数据集构建了类 Unix 操作系统（包含 Linux BSD MacOS）bash 命令与自然语言数据对的语料库。

在本项目中为了更加贴合现代对话模型微调的任务,我们将该语料库整理为 ChatML 格式, 按照 8:1:1 的标准比例划分为训练集、验证集、测试集。

2 GitHub Repo

本项目以 MIT 协议开源所有代码, 已经发布在 github 上, 具体仓库地址如下:
<https://github.com/flyingbucket/SmartMan>

3 项目说明文档

3.1 项目简介

SmartMan 是一个从微调到落地的 NL2Bash 项目, 提供完整的微调训练链路、评测流程, 以及使用 Rust 语言实现的本地命令行客户端。项目的核心目标是将自然语言 (Natural Language) 描述稳定、准确地转换为可执行的 Bash 命令, 并支持微调训练、系统评估与本地高效推理部署。

3.2 核心特性

- **QuickTune 微调框架**: 提供轻量且可扩展的微调注册与配方 (Recipe) 机制, 实现模型、LoRA 以及 SFT 配置的统一管理与高度复用。
- **微调 Pipeline**: 覆盖数据准备、模型训练、指标评估以及模型权重合并的完整闭环流程。
- **smartman-cli 客户端**: 基于 Rust 语言实现的交互式命令行工具 (CLI), 可无缝连接本地 LLM 推理引擎, 在终端内提供流畅的 NL2Bash 交互体验。

3.3 目录结构

项目代码仓库的整体目录结构如下所示:

Listing 1: 项目目录树结构

```
.
├── src/
│   ├── quicktune/           # QuickTune 微调框架核心代码
│   └── smartman/           # 训练与评估 Pipeline 逻辑
├── scripts/                # 数据处理、微调、评测及模型合并脚本
├── configs/                # 训练与评估的配置文件示例 (YAML)
├── data/                   # 数据目录 (包含原始数据与处理后的数据集)
└── smartman-cli/           # Rust 实现的 CLI 客户端源码
```

3.4 环境准备

推荐使用 Conda 虚拟环境进行环境隔离与依赖管理：

```
1 # 创建 Conda 环境
2 conda env create -f environment.yaml
3 # 激活 Conda 环境
4 conda activate smartman
```

或者，您也可以直接通过 pip 工具进行依赖安装：

```
1 pip install -r requirements.txt
2 pip install -e .
```

3.5 QuickTune 微调框架

QuickTune 框架通过高度解耦的注册表机制 (Registry) 来管理模型、LoRA 权重与 SFT 配置：

- `src/quicktune/core/manager.py` 提供了统一的组件注册与加载入口。
- `src/quicktune/recipes/` 目录下的可用配方 (Recipes) 会在运行时被自动扫描并加载。

在启动训练时，用户只需在配置文件中通过名称引用特定的 recipe，即可实现不同训练策略的高效组合与快速复用。

3.6 微调 Pipeline

项目的训练与评估主入口位于 `src/smartman/main.py`，该脚本当前支持 `train`、`eval` 以及 `all` 三种运行模式。

3.6.1 配置示例

用户可参考 `configs/example.yaml` 配置文件，在其中定义以下核心要素：

- 基座模型标识符 (`model_id`)
- 训练与评估数据集路径 (`data_dirs`)

- QuickTune 配方名称 (recipe)
- 评估细节配置 (eval_conf)

3.6.2 训练与评估指令

通过指定 `--mode` 参数来切换不同的运行阶段：

```
1 # 仅执行微调训练
2 python -m smartman.main --config configs/example.yaml --mode
   train
3
4 # 仅执行模型评估
5 python -m smartman.main --config configs/example.yaml --mode eval
6
7 # 顺序执行完整训练与评估流程
8 python -m smartman.main --config configs/example.yaml --mode all
```

3.6.3 关键脚本说明

`scripts/nl2bash.py` 构建英文 NL2Bash 基础数据集。

`scripts/make_chinese_data.py` 生成中文 NL2Bash 针对性训练数据。

`scripts/merge_data.py` 合并中英文训练集，构建混合语料。

`scripts/tune.py` 独立的微调脚本示例。

`scripts/test_model.py` 推理测试与 LoRA 权重动态加载测试。

`scripts/merge_model.py` 将训练得到的 LoRA 增量权重合并至基座模型。

3.7 评估指标

评估流程集成了多维度的自然语言转命令指标，以全面衡量模型性能：

- **S-EM (Strict Exact Match)**: 严格准确匹配率
- **BLEU**: 双语互译孤立评测指标
- **Syntax Pass**: Bash 语法通过率
- **Soft-EM F1**: 宽松准确匹配 F1 值
- **Edit Similarity**: 编辑距离相似度
- **Length Ratio**: 生成文本与目标文本长度比

评估生成的详细报表与结果将自动保存至 `eval_results/` 目录中。

3.8 smartman-cli (Rust 客户端)

smartman-cli 旨在提供原生的本地交互式终端体验，默认接入本地的 llamafire --server 服务。

3.8.1 安装指南

推荐从 GitHub Release 页面获取对应操作系统的预编译版本进行安装。Release 包含一个默认的 llamafire-thin 引擎（体积小，仅支持 CPU 推理）。

若需要更强劲的性能，可从 llamafire 官方仓库下载完整版引擎（支持更多算子与硬件加速），并通过 CLI 的安装命令将其接入。量化后的 GGUF 模型文件可从本仓库的 Release (ModelRelease) 中下载。

3.8.2 引擎与模型安装命令

使用 CLI 将引擎与模型安装并链接至本地 assets 目录：

```
1 # 安装完整版 llamafire 引擎（路径指向官方下载的引擎）
2 smartman-cli install engine /path/to/llamafire --link
3
4 # 安装量化 GGUF 模型（路径指向本仓库下载的模型）
5 smartman-cli install model /path/to/model.gguf
```

3.8.3 交互式使用说明

- **智能转换**：输入自然语言指令，客户端将实时返回转换后的 Bash 命令。
- **便捷操作**：支持直接在终端内执行生成的命令，或一键复制到系统剪贴板。
- **Shell 穿透**：在指令前加上 ! 前缀，可直接穿透执行本地的常规 Shell 命令。

3.9 开源协议

本项目基于 MIT 许可证开源。

4 测评报告

详见 report.pdf 报告 slides 的”评估指标 & Benchmark”部分。

5 团队分工

- 李孟霖: quicktune 框架及微调 pipeline 设计，落地软件 smartman-cli 开发
- 屈阳: 数据整理和数据清洗，LoRA 微调设计
- 刘治权: SFT 微调及量化参数设计，软件测试
- 赵浦渊: 数据清洗与 slides、报告撰写

参考文献

- [1] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. Nl2bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation LREC 2018, Miyazaki (Japan), 7-12 May, 2018.*, 2018.